Quantus

A Double-Bottom “Flush-Reclaim” Chart-Pattern Scanner & Alert Bot
Design, Backend Architecture & Trading Concepts — explained from scratch

Technical Design Document — generated 2026-07-28

Quantus watches a list of stocks on the daily and weekly timeframes and looks for one specific chart setup: a **double-bottom “flush-reclaim”** (a bear trap). When it finds one, it sends you a message on Telegram and Discord — with a labelled price chart, a suggested entry, stop, and target, and a position size. It does **not** place trades; a human decides. This document explains what all of that means, starting from “what is a candlestick,” so a reader new to trading and charts can follow the whole design.

Nothing here is financial advice. Quantus is a personal research and alerting tool.

1. What Quantus Is (in plain terms)

Quantus is a personal, single-user program that **scans stock charts and alerts you** when a particular bullish reversal setup appears. Think of it as a tireless assistant that watches your watchlist every day and taps you on the shoulder — with a chart and the key numbers — when something worth a look shows up. You still make every trading decision yourself.

At a glance

- **Data:** free end-of-day price history from Yahoo Finance (via the `yfinance` library). No paid data feed.
- **Watchlist:** a blend of ~57 liquid US names — a sector-diverse large-cap core plus a high-volatility sleeve (e.g. PLTR, COIN, RIVN, HOOD, FOXA) — stored in a small database and editable live from Telegram. The volatile sleeve supplies the sharp sell-offs the setup depends on (which calm large-caps rarely produce), while the core adds breadth and market-regime diversity for the walk-forward test.
- **Timeframes:** daily and weekly, scanned separately (a weekly signal is rarer and stronger).
- **The setup:** a double-bottom *flush-reclaim* — explained step by step in Sections 2–5.
- **Each alert carries:** a labelled candlestick chart, the entry / stop / target, the reward-to-risk ratio, and a suggested position size.
- **Scheduling:** a once-a-day scan run automatically; plus an always-on helper so the Telegram buttons respond instantly.
- **Honesty built in:** a backtester and a parameter sweep measure whether the idea actually works on history, with an out-of-sample check that guards against fooling ourselves.

The code is Python 3.11 (the `ddbot` package) with 120 automated tests. Every part sits behind a clean interface (data source, alerter, pattern, store) so pieces can be swapped without rewrites. **Important framing:** as the backtesting section shows, this strategy does not yet have a *proven* mechanical edge — so Quantus is used as a **discretionary finder** (it finds candidates; you judge each chart), not an auto-trader.

2. Charts 101: How to Read the Pictures

Everything Quantus does is built on the **candlestick chart**. If you already read charts, skim this; if not, this section gives you the whole vocabulary the rest of the document uses.

2.1 What a candlestick is

Each candlestick summarises the price action for one period of time (one day on the daily chart, one week on the weekly). It records four prices: the **open** (first trade of the period), the **high** (highest price reached), the **low** (lowest), and the **close** (last trade). The thick part is the **body** (open–to–close); the thin lines above and below are the **wicks** (also called shadows or tails), reaching to the high and the low. By convention the candle is **green** when price closed higher than it opened (buyers won the period) and **red** when it closed lower (sellers won).

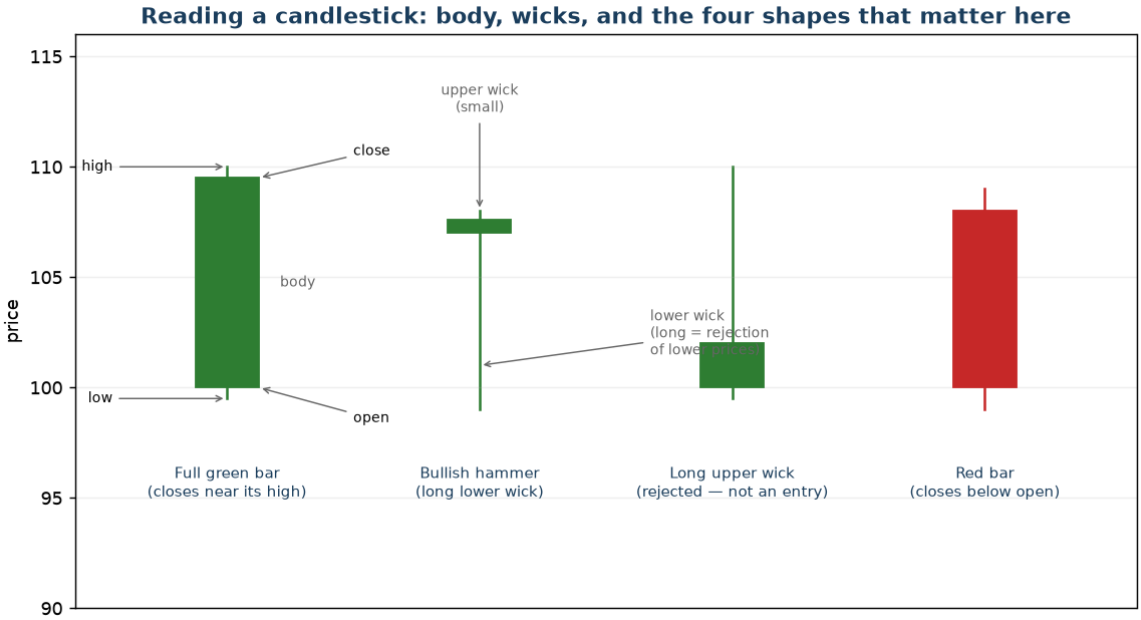

Figure 1. A candlestick records four prices (open, high, low, close). The body is open-to-close; the wicks reach to the high and low. Four shapes matter for this strategy: a full green bar (closes near its high), a bullish hammer (long lower wick = buyers rejected lower prices), a long-upper-wick bar (a rally that got sold off before the close — NOT a clean entry here), and a red bar.

Why the wicks matter so much here: a long **upper** wick means price pushed up during the period but was **sold back down** before the close — a sign of sellers overhead. A long **lower** wick (a “hammer” or “pin bar”) means price dropped but buyers **rejected** the lower prices and pushed the close back up — a bullish sign. Quantus only treats a candle as a valid entry trigger if it is a clean bullish shape (a full green body or a hammer) with a *small* upper wick.

2.2 Support, resistance, and trend

Support is a price level where buyers have repeatedly stepped in and stopped a decline (a “floor”). **Resistance** is the mirror image: a level where sellers repeatedly cap a rally (a “ceiling”). A **trend** is the general direction — a downtrend is a series of lower lows. These three words are all we need: the strategy is about price falling into a support area, being pushed below it in a trap, and then reversing.

3. The Strategy’s Ideas, One at a Time

3.1 The double bottom and the neckline

A **double bottom** is a bullish reversal shaped like the letter “W.” After a decline, price makes a low (**Bottom 1**), bounces up to an interim **peak**, falls again toward the same area, and eventually turns back up. The two bottoms mark a support level that buyers defended. The interim peak is the **neckline** — the resistance ceiling between the bottoms. In the classic version, you buy when price breaks *above* the neckline. Quantus uses a sharper variation, below.

3.2 The flush-reclaim (a “bear trap”) — the actual entry

Instead of waiting for a breakout above the neckline, Quantus enters **below** the neckline, on a failed breakdown. The sequence:

- Price falls into **Bottom 1** after a prior decline (there must be a downtrend to reverse).
- It **recovers** up toward an interim peak — that peak becomes the **neckline** / target.

- Then comes a **steep, high-volume flush**: a near-vertical sell-off that **undercuts** Bottom 1 (pokes below the old floor). This is the **bear trap** — it looks like a fresh breakdown and flushes out nervous sellers.
- Within a few bars a **clean bullish candle reclaims** back above Bottom 1's low (but still below the neckline). That failed breakdown is the signal: sellers had their chance and could not hold the lows. **That reclaim candle's close is the entry.**

The appeal is buying the reversal **cheaply** (well below the neckline) right as the trap springs, rather than chasing a breakout. The risk is that the "trap" is a real breakdown — which is what the stop is for.

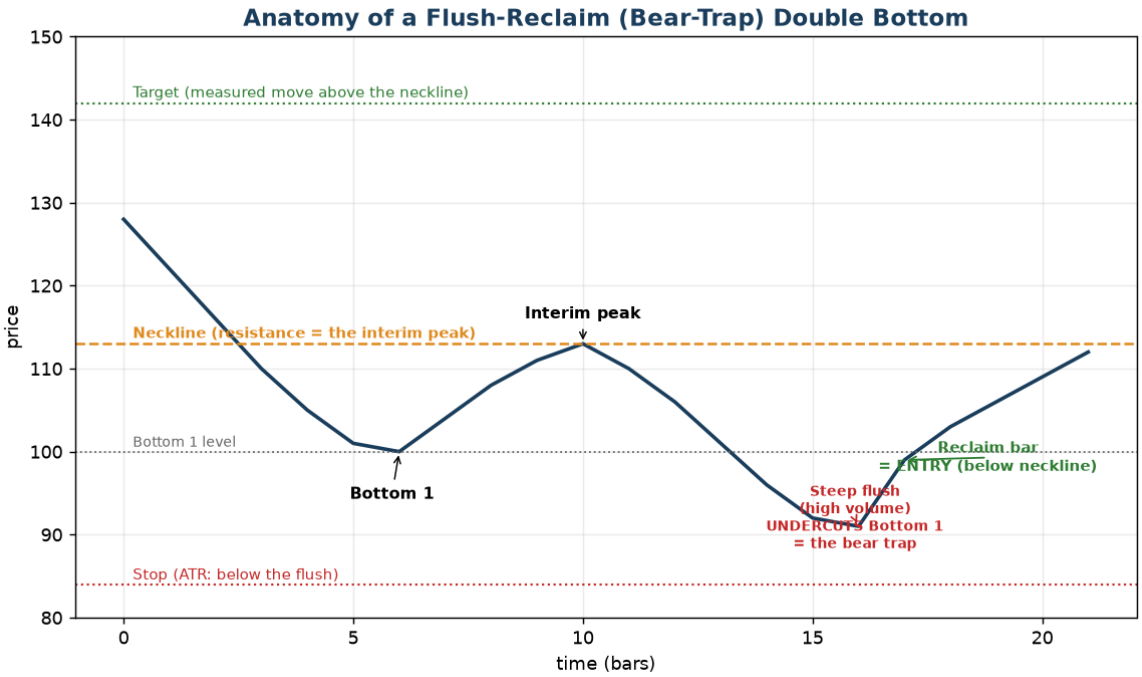

Figure 2. The flush-reclaim. Price declines to Bottom 1, recovers to the neckline, then a steep high-volume flush undercuts Bottom 1 (the bear trap). A clean bullish candle reclaims back above Bottom 1 = the entry, below the neckline. The stop sits below the flush; the target is a 'measured move' above the neckline.

3.3 Finding it objectively: swing points and prominence

To turn this picture into code, the bottoms are found as **swing lows**: a bar is a swing low if its low is the lowest within a symmetric window of `swing_k` bars on each side. **Prominence** then measures how far the neckline sits above the bottom, as a percentage; requiring a minimum (5%) throws away shallow, sideways noise and keeps setups with real depth.

3.4 Measuring "steep" and "capitulation": ATR and volume

Two things make the flush a genuine trap rather than a gentle dip. First, **steepness**, measured with the **ATR** (Average True Range) — a standard gauge of how much a stock typically moves per bar (its volatility). Requiring the drop from the peak to be at least a multiple of ATR (e.g. 3×) means "much bigger than a normal move" — a near-vertical plunge. Second, **volume**: the flush bar must trade well above its recent average volume, the fingerprint of panic selling (**capitulation**).

3.5 The clean reclaim bar (why the candle shape matters)

The entry candle must be a **clean bullish shape** so we are not buying a half-hearted bounce. It must be green and either (a) a **full body** that closes near its high, or (b) a **hammer** with a long lower wick — and in both cases the **upper wick must be small** (the close lands in the top 15% of the bar's range). A green candle that rallied but got **sold off** into the close leaves a long upper wick; Quantus rejects it, because that

overhead selling is exactly what a reversal entry does not want.

3.6 Only closed bars (no “repainting”)

Quantus acts only on **completed** candles; the still-forming current bar is ignored, because it can change (“repaint”) before it closes. This discipline is what makes a backtest honest: the bot never reacts to information that did not fully exist yet.

3.7 Keeping score: R-multiples, expectancy, profit factor, drawdown

- **R (one unit of risk):** the distance from your entry to your stop. If a trade makes twice what it risked, that is +2R; hitting the stop is -1R. Measuring in R lets you compare trades across different stocks and sizes.
- **Expectancy (average R):** the mean R per trade — your average result per signal. Positive expectancy is the definition of an edge.
- **Win rate:** the share of trades that make money. A high win rate with tiny average R is not automatically good; the two must be read together.
- **Profit factor:** total winning R divided by total losing R. Above 1.0 is profitable; ~2+ is strong.
- **Max drawdown:** the deepest peak-to-trough dip of the running total — the worst stretch you would have had to sit through.
- **Statistical significance:** with only a handful of trades, even a good average can be luck. A **bootstrap** (resampling the trades thousands of times) gives a 95% confidence interval for the average R; only when that interval’s lower bound stays **above zero** is the positive edge unlikely to be noise. A small sample is flagged explicitly so a shiny number is never mistaken for proof.

3.8 Not fooling ourselves: walk-forward and out-of-sample

Overfitting is tuning a strategy until it looks perfect on the past, then watching it fail live. Three defenses are built in. **Walk-forward** testing replays history bar by bar, only ever using data that existed at each moment. **Out-of-sample** testing hides the most recent slice of history, tunes on the older part, and then checks whether the choice still works on the hidden slice. If an edge vanishes out-of-sample, it was a mirage and is thrown away. (This exact check is what told us the original exit rule did not work — see Section 7.)

A third, added most recently, is **rolling walk-forward analysis**: rather than one hidden slice, it cuts the whole history into several *sequential* time windows and scores the current strategy in each one independently. A single good average can hide the truth that the edge lived entirely in one lucky stretch; splitting into windows shows whether the result **repeats across different market regimes**. Crucially, each window also reports how many trades it contains — a window with a handful of trades is flagged as **low-sample**, so a shiny number backed by three trades is never mistaken for evidence. This is what keeps the ‘is there really an edge?’ question honest as the strategy evolves (see Section 7.3).

4. Backend Architecture

The system is a set of small, single-purpose modules connected in a straight line. The **same** detection code powers both the live scanner and the backtester, so what we measure is what we trade.

4.1 Data-flow pipeline (the daily scan)

```
scheduled daily scan
      |
      v
  ddbot.run  ->  YahooDataProvider          (fetch completed OHLCV bars)
      |            |
      |            v
      |         detect()                     (swing lows -> recovery/neckline ->
      |            |                          steep undercut flush = candidate)
      |            v
      |         dedupe_by_neckline()         (one setup per resistance level)
      |            |
      |            v
      |         PatternStore (SQLite)        (remember pending setups between runs)
      |            |
      |            v
      |         check_confirmation()         (clean reclaim below neckline? -> set
      |            |                          entry + ATR stop + measured target)
      |            v
      |         multi-timeframe gate -> chart -> Telegram + Discord
      v
   exit (idempotent: each confirmed setup is alerted exactly once)
```

4.2 Module map

Module	Responsibility
data/yahoo.py	Fetch price history from Yahoo; drop the still-forming bar; validate symbols.
patterns/swings.py	Swing-low / swing-high (fractal) detection.
patterns/double_bottom.py	detect(); dedupe_by_neckline(); check_confirmation(); compute_stop()/compute_target().
patterns/base.py	The DoubleBottom record (entry, stop, target), states, stable id.
state/store.py	SQLite: setups (incl. stop/target), watchlist, key/value. Migrates old DBs.
risk.py	Position sizing: how many shares to risk a fixed % of the account.
mtf.py	Multi-timeframe filter (only take a daily signal if the weekly trend agrees).
alerts/*	Telegram, Discord, fan-out, and the message formatter.
charts/chart.py	The labelled candlestick PNG attached to each alert (mplfinance).
engine.py	Runs fetch -> detect -> remember -> confirm -> alert per ticker/timeframe.
journal.py	Replays past alerts against later prices to score how they actually did.
backtest/*	Walk-forward engine, metrics, parameter sweep, rolling walk-forward folds, CLI.

4.3 Memory and “idempotency”

All state lives in one SQLite file. Each setup gets a stable id (a hash of the ticker, timeframe, and the two bottom dates). Because a setup often confirms on a *later* day, setups persist between runs; an `alerted` flag guarantees each one fires **exactly once**. That makes every scheduled run safe to repeat — “idempotent.” When the exit model was added, the database automatically gained two new columns (the stop and target prices) via a small migration, so existing data keeps working.

5. The Detection Algorithm, Step by Step

Given a set of completed price bars for one ticker and timeframe:

- **1. First bottom (B1):** find a swing low, and require a **prior downtrend** into it.
- **2. Recovery / neckline:** require a bounce to an interim peak at least `min_prominence_pct` above B1. That peak is the neckline (and the target).
- **3. Steep flush (B2):** a near-vertical drop that **undercuts B1's low** within `flush_max_bars`, where the fall is at least `flush_atr_mult`×ATR **and** the flush bar's volume is well above its recent average (capitulation).
- **4. Dedup:** collapse near-identical necklines so one trap is one setup.
- **5. Remember:** store the surviving structures as pending.
- **6. Reclaim = entry:** within `reclaim_window` bars, the first **clean bullish candle** (full green or hammer, small upper wick) that closes above B1's low but below the neckline confirms the setup. The entry is that close; the **stop** and **target** are computed here (Section 3.4–3.5 and 7).
- **Invalidation:** a close back below the flush low, a reclaim that overshoots above the neckline (that's a plain breakout, not our below-neckline entry), or the reclaim window elapsing.

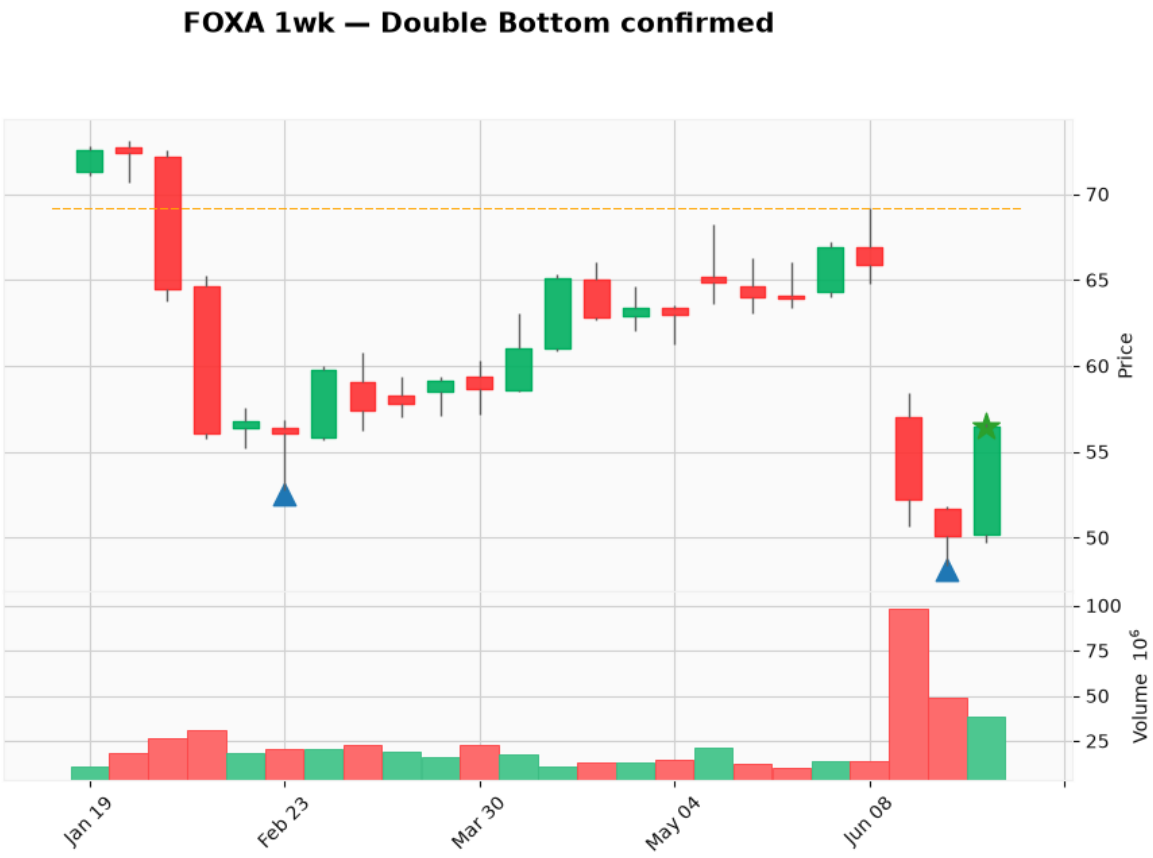

Figure 3. A real flush-reclaim Quantus detected (FOXA, weekly). The blue markers are the two bottoms (the second undercutting the first), the dashed orange line is the neckline, and the star marks the confirming reclaim candle. This is the exact image attached to the alert.

5.1 Tunable thresholds (config/config.yaml)

Everything is a named knob in one config file, so the strategy can be tuned without touching code.

Parameter	Default	Meaning

lookback_bars	90	How many recent bars are scanned per ticker.
swing_k	3	Bars each side needed to qualify a swing low.
min_prominence_pct	0.05	Neckline must sit >= 5% above Bottom 1 (real depth).
max_bars_between	50	Max bars from Bottom 1 to the flush.
require_undercut	true	The flush must poke BELOW Bottom 1's low (the trap).
flush_atr_mult	3.0	Peak->flush drop must be >= 3x ATR (a near-vertical plunge).
flush_atr_window	14	Bars used to measure ATR (typical move size).
flush_max_bars	3	The flush must happen within this many bars.
flush_volume_factor	1.5	Flush-bar volume >= 1.5x its recent average (capitulation).
reclaim_window	6	The reclaim must occur within this many bars of the flush (see 7.5).
reclaim_min_body_frac	0.60	Full green bar: body >= 60% of the bar's range.
reclaim_max_upper_wick_frac	0.15	Upper wick <= 15% of range (no 'long head').
reclaim_min_lower_wick_frac	0.50	Hammer: lower wick >= 50% of range.
stop_mode / stop_tick	swing_low / 0.01	Stop = flush (B2) swing low - one tick (see Section 7).
target_mode / target_r_multiple	r_multiple / 1.85	Target = entry + 1.85 x (entry - stop).

A note on tooling: multi-bar chart patterns like this are **not** provided by off-the-shelf libraries such as TA-Lib (which cover indicators and single-candle patterns). The structural detection here is custom logic built on NumPy and pandas.

6. Alerts, Charts, Interaction & Scheduling

6.1 What an alert looks like

On confirmation the bot renders a labelled candlestick chart (the two bottoms, the neckline, and the reclaim candle marked) and sends it, with the message, to Telegram and Discord at once; a failure on one channel never blocks the other. The message states the entry (reclaim close), the neckline, and **two stop options** the trader chooses between — a **swing-low stop** (one tick below the flush low) and a **1×ATR stop** — each shown with its own **R-multiple target** (default 1.85R) and **reward-to-risk**, plus a **suggested position size** (shares to risk a fixed 1% of a configured account, sized off the primary swing-low stop). A footnote reminds you it is a discretionary setup to review by eye.

6.2 Multi-timeframe agreement

A daily signal is stronger when the bigger picture agrees. With the multi-timeframe filter on, a **daily** alert only fires if the **weekly** trend is up (weekly close above its moving average). Weekly signals are not gated — they are the higher timeframe.

6.3 Editing the watchlist from Telegram

The watchlist lives in the database and is editable from your phone via two buttons ([+ Add] / [- Remove]) and the commands /list, /add SYMBOL, /remove SYMBOL. New symbols are checked against Yahoo before being added, and only your own chat is allowed to make changes. Because Telegram cannot wake a stopped program, a small always-on listener (auto-restarting) makes the buttons respond instantly.

6.4 Scheduling and a macOS lesson

The daily scan runs once a day, automatically. Because the bot only acts on completed bars, the exact minute does not matter. One practical lesson: macOS privacy protection blocks scheduled agents from running scripts kept in the Desktop folder; the fix was to move the project out of Desktop and run the Python interpreter directly.

7. Does It Actually Work? Backtesting & the Exit Study

7.1 How the backtest works

The backtester replays history through the **exact same** detection code the live bot uses, walking forward bar by bar so it never peeks at the future. The first bar a setup reclaims is the entry; the trade is then followed until it hits its stop (a loss) or its target (a win), or times out. Every result is recorded in R (Section 3.7). Caveats: the model ignores trading costs, slippage, and dividends, and takes one position per signal — so results are indicative, not a promise.

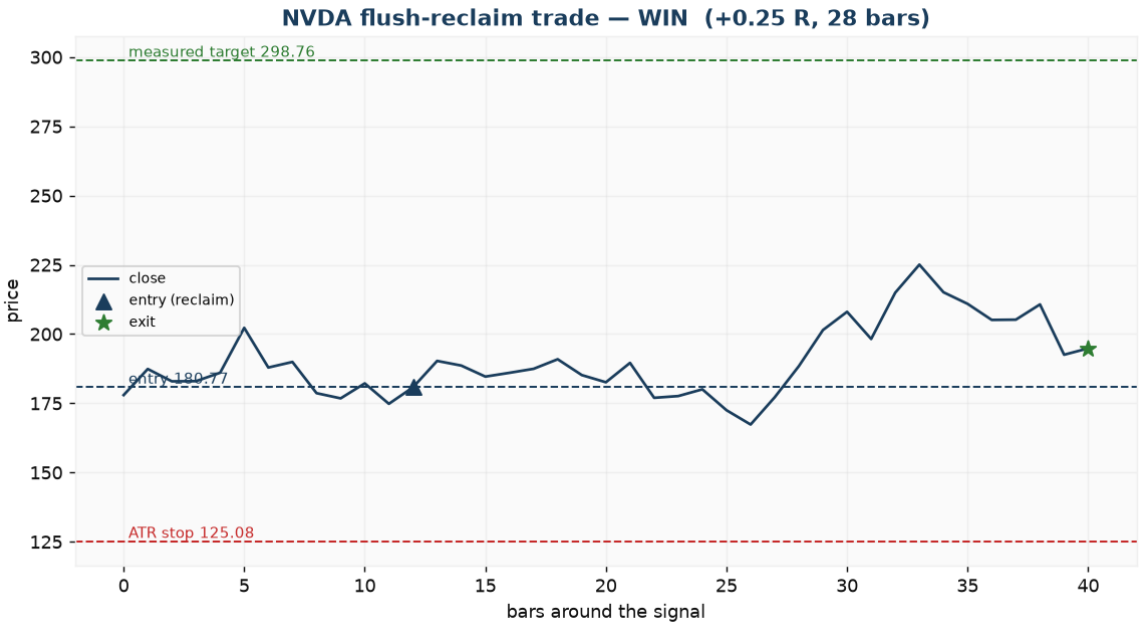

Figure 4. One backtested trade. Entry at the reclaim close, an ATR-based stop below, and a measured-move target above; the star marks the exit. The outcome is recorded in R (reward relative to the risk taken).

7.2 The key finding: the exit was the problem

The first version of the strategy stopped out just below the deep flush and aimed only at the neckline. Backtesting showed the entry was fine (~60% of trades were winners) but the strategy still **lost money out-of-sample**: the stop was so far away that the few losses outweighed the many small wins. So a dedicated **exit-model study** swept different stop and target rules across ~36 volatile stocks over ~8 years, splitting every result into in-sample and out-of-sample.

Stop → Target	In-sample	Out-of-sample	Verdict
ATR (3.5x) → measured move	+0.62 R, PF 2.8	+0.71 R, PF 4.5	Holds out-of-sample
ATR (3.5x) → 2R target	+0.53 R, PF 2.4	+0.87 R, PF 5.3	Holds out-of-sample
Flush low → neckline (original)	+0.04 R, PF 1.1	-0.19 R, PF 0.6	Fails (negative OOS)
Reclaim-bar low (tight) → any	negative	negative	Worst (whipsawed out)

The lesson: a **wider, volatility-scaled ATR stop** (giving the trade room) with a **measured-move target** turns a losing exit into one that stays profitable out-of-sample — positive with a profit factor above 2 in *both* samples, the signature of a real (not curve-fit) effect. A very tight stop was the worst: it got shaken out by normal wobble. That ATR/measured-move exit is now the live default. **Honest caveat:** the setup is rare, so this rests on only ~16 out-of-sample trades — promising, not proof. That is exactly why Quantus stays a discretionary finder.

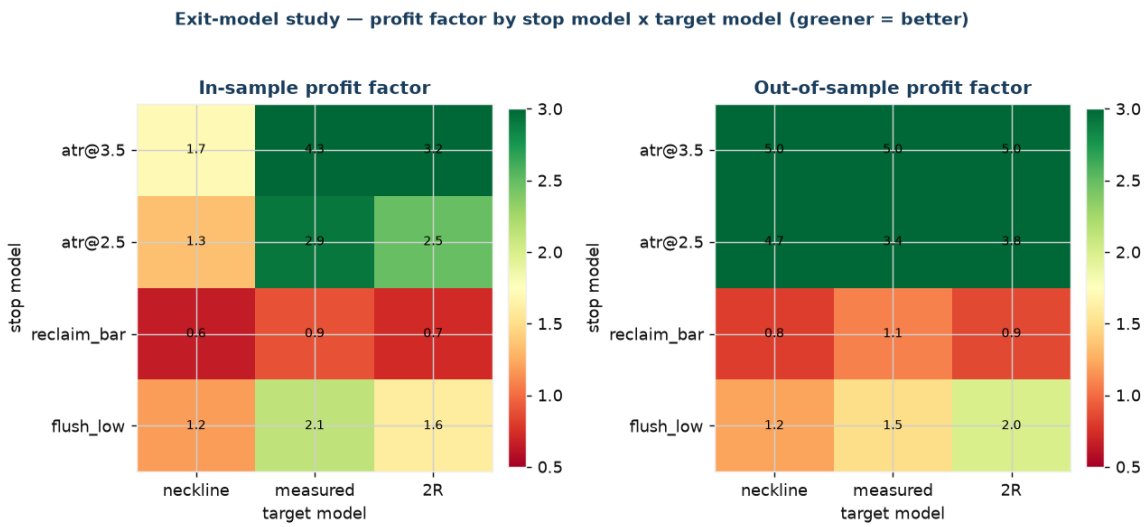

Figure 5. Profit factor for every stop-model (rows) x target-model (columns) combination, in-sample (left) vs out-of-sample (right); greener is better. The ATR-stop rows stay green in both panels; the tight reclaim-bar stop is red; the original flush-low/neckline corner fades. Only settings green in BOTH panels are trustworthy.

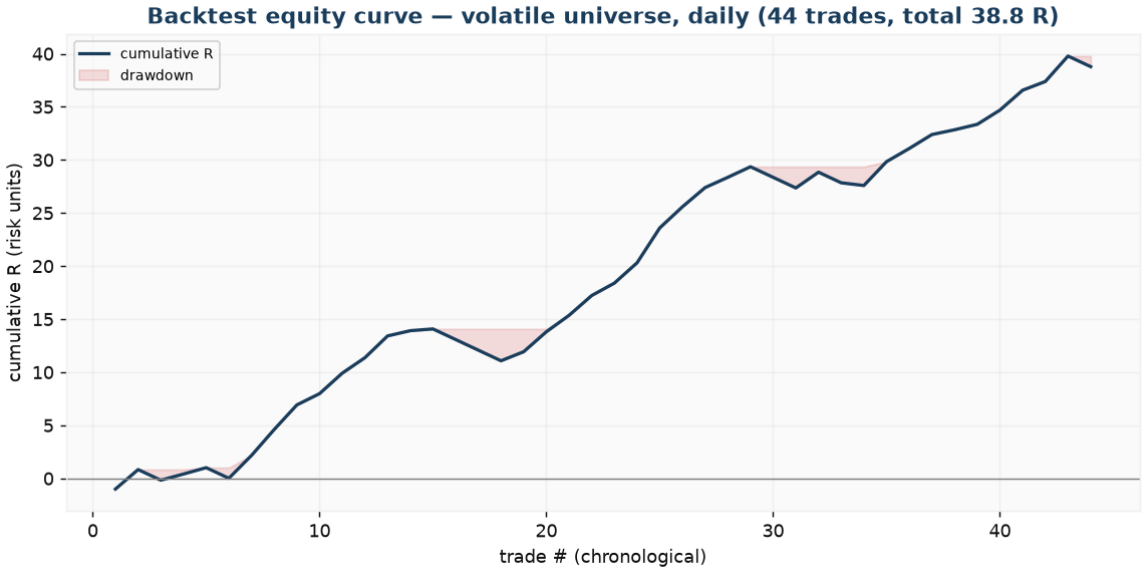

Figure 6. The running total (in R) of the backtested trades across the volatile universe, in date order. The shaded band is drawdown — how far below its prior peak the running total sat at each point.

7.3 Is the edge stable over time? Rolling walk-forward

A single all-history average, however good, can be produced by one exceptional stretch surrounded by mediocrity. To test for that, the backtester can slice the trades into several **sequential time windows** (`--walk-forward N`) and score the current live configuration in each window on its own, alongside a held-out newest slice. A trustworthy edge shows up as **most windows staying positive** — not one big winner carrying the rest.

Run today, the result is **encouraging in direction but not yet conclusive**: the majority of windows are positive and the held-out newest slice is positive, but **every window is flagged low-sample**. The reason is the setup's rarity — the filters are strict enough that even several years across a broad watchlist yield only a handful of trades per window. That is an honest, useful finding rather than a disappointment: it says the direction of the edge is right, but there are simply too few trades to call it proven — which is precisely why Quantus remains a **discretionary finder**, and why raising the trade count (a more volatile watchlist, or carefully relaxing the most restrictive filters) is the next question worth studying.

To support exactly that study, the **parameter sweep** (Section 3.8) grids not only the detection thresholds but also the **exit model** (stop and target rules), splits every combination into in-sample and out-of-sample, and flags whether each one's out-of-sample edge is **statistically significant** (its bootstrap confidence interval on average R stays above zero). That makes it easy to see which settings genuinely survive, rather than which merely looked best on the tuning data.

7.4 Exit-model refinement: swing-low / 1×ATR stops with an R-multiple target

A follow-up study compared the earlier default against the two stops a discretionary trader actually uses, each paired with a fixed **1.85R** target: a **swing-low stop** (one tick below the flush low) and a **1×ATR stop**. Because the exit model does not change which signals fire, all three were measured on the same trades. Both trader methods **beat** the earlier 3.5×ATR / measured-move exit: higher average R and profit factor, and — the point that matters — a **higher confidence-interval floor** (more statistically robust, not less). The capped R-multiple target is why: it turns every win into a clean +1.85R instead of a variable measured move. The swing-low stop is now the **canonical** (stored, journalled, position-sized) exit, and each alert additionally shows the 1×ATR option so the trader picks per trade. The low-sample caveat from 7.3 still applies — better, not yet proven.

7.5 Clearing the sample-size bar: the reclaim window

The recurring limitation through 7.1–7.4 was **too few trades** — roughly two dozen over several years, below the ~30 needed to trust the statistics. A sweep of the tightest filters (each ranked by out-of-sample edge and flagged for significance) found one clean lever: the **reclaim window**. Widening it from **4 to 6 bars** — how long the bullish reclaim may take to form after the flush — lifted the trade count across the watchlist from ~24 to **31**, finally clearing the adequacy bar, while the edge held: profit factor unchanged (~3.2), average R barely moved (+0.73 to +0.68R), the bootstrap confidence interval stayed above zero, and **every** rolling walk-forward window turned positive (one had been slightly negative). It works because the reclaim window is a **patience** setting, not a quality filter: a wider window admits genuine setups that simply form a bar or two slower, rather than lower-quality ones. Loosening the steepness or depth filters instead (e.g. a 2×ATR flush) added more trades but **destroyed** the edge — so 6 is the sweet spot, and is now the default. This is the first configuration that is both statistically significant *and* adequately sampled.

8. How the Project Evolved

Quantus was built and revised incrementally, each step verified with tests and live runs. The most important changes were driven by evidence, not opinion.

Stage	What changed
MVP	Daily double-bottom detect + confirm; Telegram + Discord alerts; SQLite memory; idempotent daily run.
Expansion	Weekly timeframe; labelled chart images; automatic scheduling; watchlist in the database.
Telegram control	Buttons + commands to edit the watchlist; an always-on listener for instant replies.
Quality filters	One-alert-per-setup dedup; volume confirmation; multi-timeframe agreement.
Backtesting	Walk-forward engine, metrics, and a parameter sweep with an out-of-sample guard.
Risk + journal	Suggested position size on every alert; a journal that scores how past alerts played out.
Strategy pivot	Replaced the breakout entry with the flush-reclaim (bear-trap) entry below the neckline.
Cleaner entries	Required the reclaim candle to be a clean bullish shape (full green / hammer, small upper wick).
Exit-model study	Found the flush-low/neckline exit was negative out-of-sample; adopted an ATR stop + measured-move target that
Digest + health	Weekly digest of live results, pushed to Telegram/Discord; scan/listener heartbeats warn on silent failure.
Walk-forward validation	Rolling multi-window walk-forward with per-window low-sample flags; confirmed the edge's direction holds but trad
Exit refinement	Added swing-low & 1xATR stops with a 1.85R target; both beat the old exit OOS. Swing-low is canonical; alerts sl
Sample size	Widened the reclaim window 4->6 bars; trades cleared the ~30 adequacy bar (24->31) with the edge intact and ev
This document	Rewritten to explain the current strategy from scratch; updated with the walk-forward and exit-refinement findings.

9. Risks & Limitations

- **No *proven* mechanical edge yet.** The improved exit looks good but rests on a small out-of-sample sample. Treat every alert as a candidate to review, not a signal to trade blindly.
- **False positives are inherent** to pattern detection; the prominence, steep-flush, volume and clean-reclaim filters reduce them but never eliminate them.
- **Data quality:** Yahoo Finance is a free, unofficial feed with no guarantees; it can be delayed, gap, or be revised. Prices are split/dividend-adjusted.
- **Modelling gaps:** the backtest ignores trading costs, slippage and dividends, and uses one position per signal — live results will differ.
- **Overfitting risk** whenever thresholds are tuned; always prefer settings that survive out-of-sample.
- **Not financial advice, and not automated:** Quantus alerts; a human decides and executes.

10. Roadmap

- **Raise the trade count:** rolling walk-forward shows the edge is directionally right but every window is low-sample. Study a more volatile watchlist and/or carefully relaxed filters to lift the signal rate without breaking the out-of-sample edge — the single most valuable next step.
- **Validate live:** let the journal accumulate real forward signals under the new exit and reconcile them against the backtest as the live sample grows.
- **More patterns:** inverse head-and-shoulders, triple bottom — reusing the swing/neckline machinery.
- **Observability (in place):** a weekly digest of live results plus scan/listener heartbeats now ship; extend with a small dashboard and longer-horizon reporting.

- **Later:** a fundamental overlay, and — only with hard guardrails — semi-automated execution.

Quantus — technical design document. Generated programmatically from docs/generate_pdf.py; regenerate to keep it in sync with the codebase.